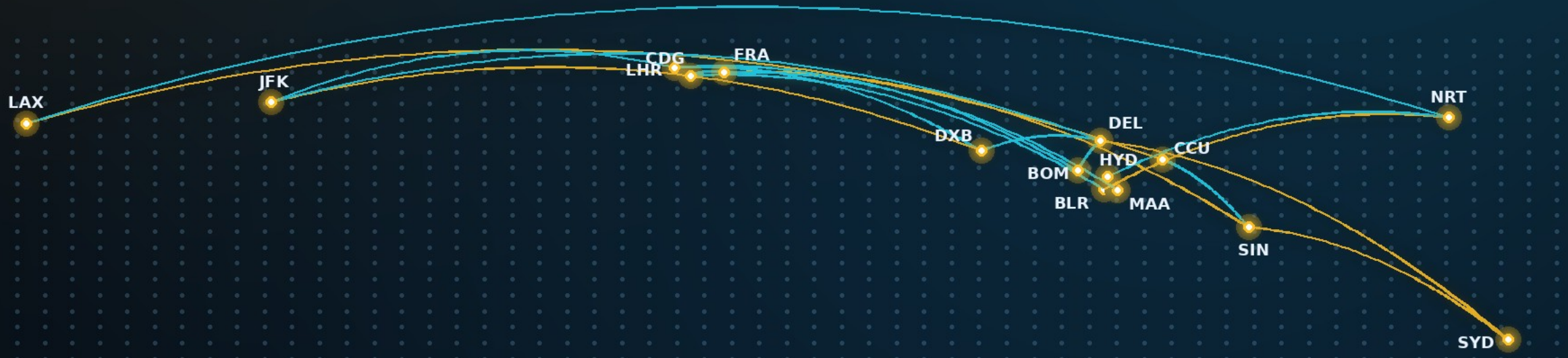
SQL PROJECT | DATA ANALYTICS

AIRLINE FLIGHT BOOKINGS ANALYSIS

From raw booking records to revenue, route and fare insights using MySQL

Ananya Gadodia
Course: Data Analytics | Tools: Python · Jupyter · MySQL Workbench · SQL



PROJECT OVERVIEW

Project Overview

This project analyzes an airline booking database spanning seven interconnected tables: airports, airplanes, seats, flights, bookings, tickets and segments. It covers 900 flights across 15 airports, 3,000 bookings, 4,805 tickets and 6,258 flight segments, all from 2025.

900
Flights

3,000
Bookings

6,258
Segments

15
Airports

Core Objective

Use SQL to turn raw flight and booking data into operational and business insights: fare and revenue patterns, airport and route performance, flight scheduling, and passenger check-in behavior.

ANALYSIS FOCUS AREAS

- **Fare & Revenue** — ₹62.37M earned
- **Airports & Routes** — 15 hubs · FRA→SIN top
- **Flight Scheduling** — 900 flights · Dec peak
- **Passenger Check-in** — 49.3% seats assigned

PROBLEM & OBJECTIVES

Problem Statement & Objectives

Airlines generate large volumes of data across flights, bookings, tickets and fares. Analyzing these interconnected datasets by hand is impractical. SQL provides a structured, repeatable way to retrieve, compare and derive insight.

- ✓ Managing interconnected booking and flight datasets
- ✓ Handling complex table relationships (PK / FK)
- ✓ Ensuring data accuracy and consistency
- ✓ Comparing fares and revenue across classes, airports and routes
- ✓ Converting SQL results into actionable insights

01 Question

02 SQL Query

03 Result

04 Insight

TECHNOLOGY

Technology Stack

DB

MySQL

Relational database engine for storing and querying all airline data.

WB

MySQL Workbench

Primary environment for writing queries, loading CSVs and reverse-engineering the schema.

7

7 Airline Tables

Structured datasets covering airports, airplanes, seats, flights, bookings, tickets and segments.

ER

EER Diagram

Reverse-engineered schema view showing keys, relationships and constraints.

SQL

SQL Analysis

Aggregations, joins, subqueries, CTEs, views and window functions.

Py

Python & Jupyter

Pandas notebooks to clean, validate and export all 7 CSV files before loading into MySQL.

DATA & DESIGN

Data Sources: 7 Relational Tables

Table	Purpose	Rows
airports	Airport codes, names, cities and countries	15
airplanes	Aircraft model, range, speed and seat capacity	10
seats	Seat map and fare class per aircraft	2,522
flights	Scheduled and actual flight times, routes and status	900
bookings	Booking reference, date and total amount	3,000
tickets	Passengers per booking and journey direction	4,805
segments	Ticket-to-flight legs with fare class, price and boarding details	6,258

Source: Kaggle airline bookings dataset. All 7 files were cleaned in Python and loaded into the MySQL database airline_booking_clean (17,510 rows in total).

Entity Relationship Diagram



One booking has many tickets; one ticket has up to two segments (1 → N). Every segment links a ticket to a flight; every flight links to two airports and one airplane; every airplane owns many seats.

METHODOLOGY

Data Flow



Raw data → Python cleaning → clean CSVs → MySQL database → SQL analysis → insights

DATA PREPARATION

Data Cleaning with Python

01
Load

read_csv on each file; IDs read as text so ticket numbers never change.

02
Profile

Shape, data types, null counts and duplicates before any change.

03
Standardize

Trim spaces, fix case and column names for every text field.

04
Missing Values

Keep meaningful NULLs (unflown flights, no check-in); never invent data.

05
Duplicates

Full-row and primary-key duplicates checked in all 7 tables.

06
Data Types

Text to datetime; boarding_no from 1.0 to whole number 1.

07
Validity & Outliers

Formats, ranges and IQR outliers checked by fare class.

08
Integrity

Every foreign key verified; orphan rows would be removed.

17,510 rows checked across 7 tables: 0 duplicates, 0 orphan keys, 0 invalid values, and 1 column type corrected.

DATA PREPARATION

Cleaning Summary by Table

Table	Rows	Missing values	Duplicates	Action taken
airports	15	0	0	Codes validated (3 letters); no change needed
airplanes	10	0	0	Ranges checked; codes read as text
seats	2,522	0	0	Composite key and fare classes validated
flights	900	456 actual-time blanks	0	Dates converted; blanks kept as NULL (flights not yet operated)
bookings	3,000	0	0	Amounts set to 2 decimals; no outliers
tickets	4,805	0	0	115 names with titles (MD, PhD) kept; outbound set to 1/0
segments	6,258	3,175 check-in blanks	0	boarding_no fixed to integer; blanks kept as NULL

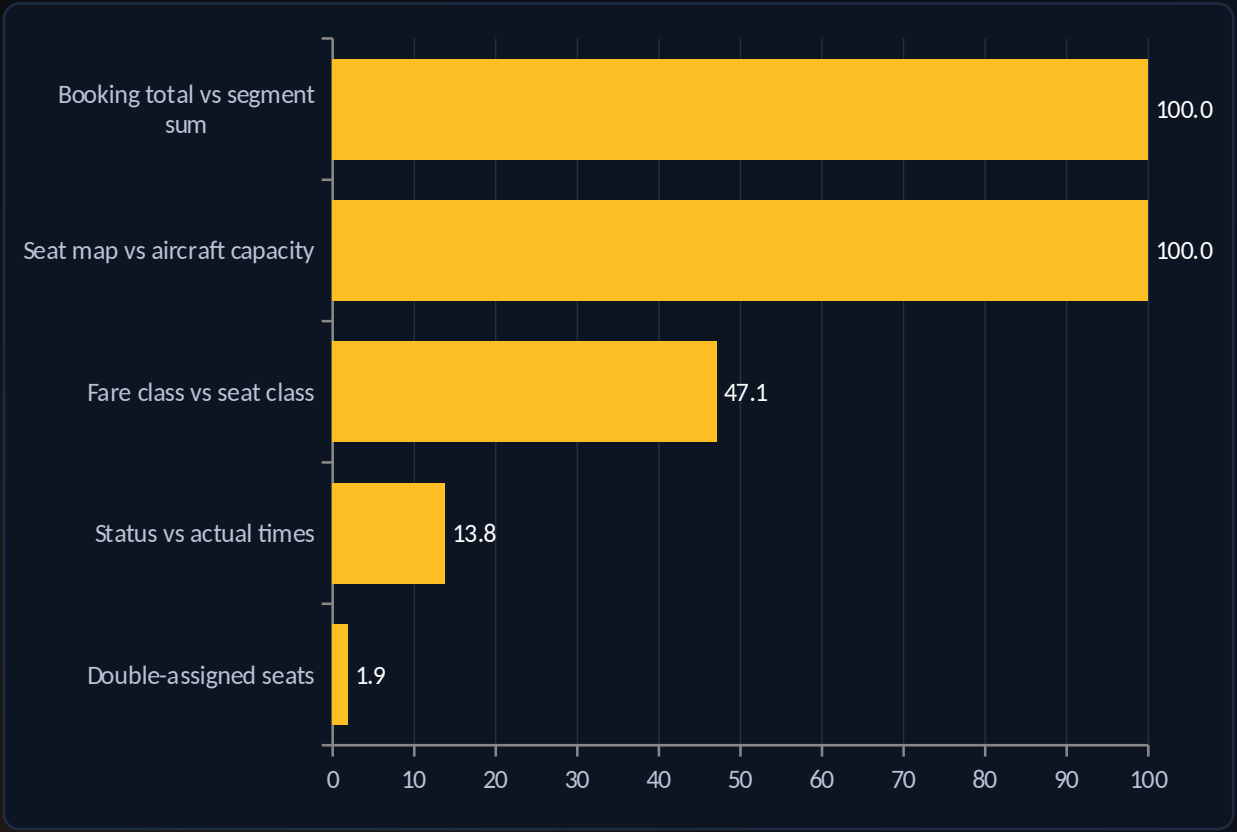


Blank values were kept as NULL where they carry meaning, so check-in and cancellation queries stay accurate.

DATA PREPARATION

Data Quality Findings

Share of records affected (%)



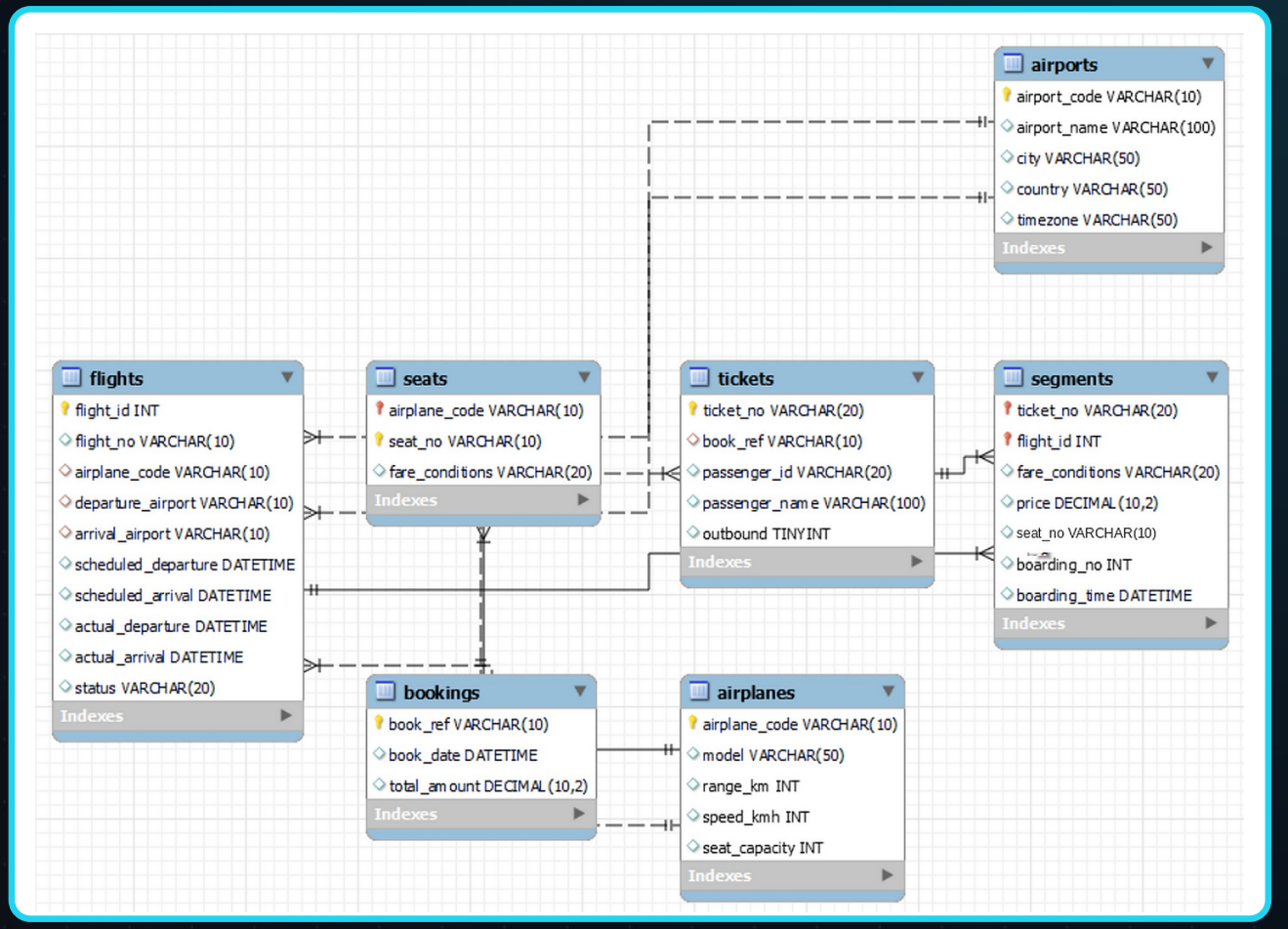
Finding	Affected	How handled
Booking totals differ from segment prices	3,000 of 3,000	Revenue uses segments.price
Seat map larger than capacity	10 of 10 aircraft	Capacity taken from airplanes
Paid class differs from seat class	1,451 of 3,083	Fare class from segments
Delayed / On Time without actual times	124 of 900 flights	Kept as recorded
Same seat on one flight twice	59 of 3,083	Kept; noted as limitation

These are source-data inconsistencies. Every query uses the field that reflects what was actually paid or flown.

DATABASE DESIGN

MySQL Workbench Reverse Engineering

Table	Primary Key	Foreign Keys
airports	airport_code	-
airplanes	airplane_code	-
seats	(airplane_code, seat_no)	airplane_code → airplanes
flights	flight_id	airplane_code, departure_airport, arrival_airport
bookings	book_ref	-
tickets	ticket_no	book_ref → bookings
segments	(ticket_no, flight_id)	ticket_no → tickets, flight_id → flights



Composite keys on seats and segments prevent duplicate seats per aircraft and duplicate legs per ticket.

METHODOLOGY

SQL Analysis Framework

1

Question

Define the business question clearly.

2

SQL Query

Construct a targeted query for retrieval.

3

Result

Execute and retrieve the structured output.

4

Comparison & Calculation

Compare groups and compute metrics.

5

Insight & Judgement

Translate findings into recommendations.

Worked example

```
SELECT fare_conditions, SUM(price) AS total_revenue
FROM segments
GROUP BY fare_conditions;
```

Question: which fare class generates the most revenue? Result: Economy ₹26.70M, Business ₹24.73M, Comfort ₹10.94M. Insight: Business is only 10.6% of segments yet earns 39.7% of revenue.

Segment Revenue at a Glance

6,258

Total flight segments

₹62.37M

Total segment revenue

₹9,966.61

Average fare per segment

₹2,500.39

Minimum fare

₹59,969.57

Maximum fare

```
SELECT COUNT(*), SUM(price),
  AVG(price), MIN(price),
  MAX(price)
FROM segments;
```

Fares span a wide band, from ₹2,500.39 to ₹59,969.57, so averages must be read alongside fare class.

Filtering & Sorting Records

1,509

2025 bookings above ₹50,000 (50.3% of 3,000)

231

Outbound passengers whose name starts with "A"

```
WHERE YEAR(book_date)=2025
  AND total_amount>50000;

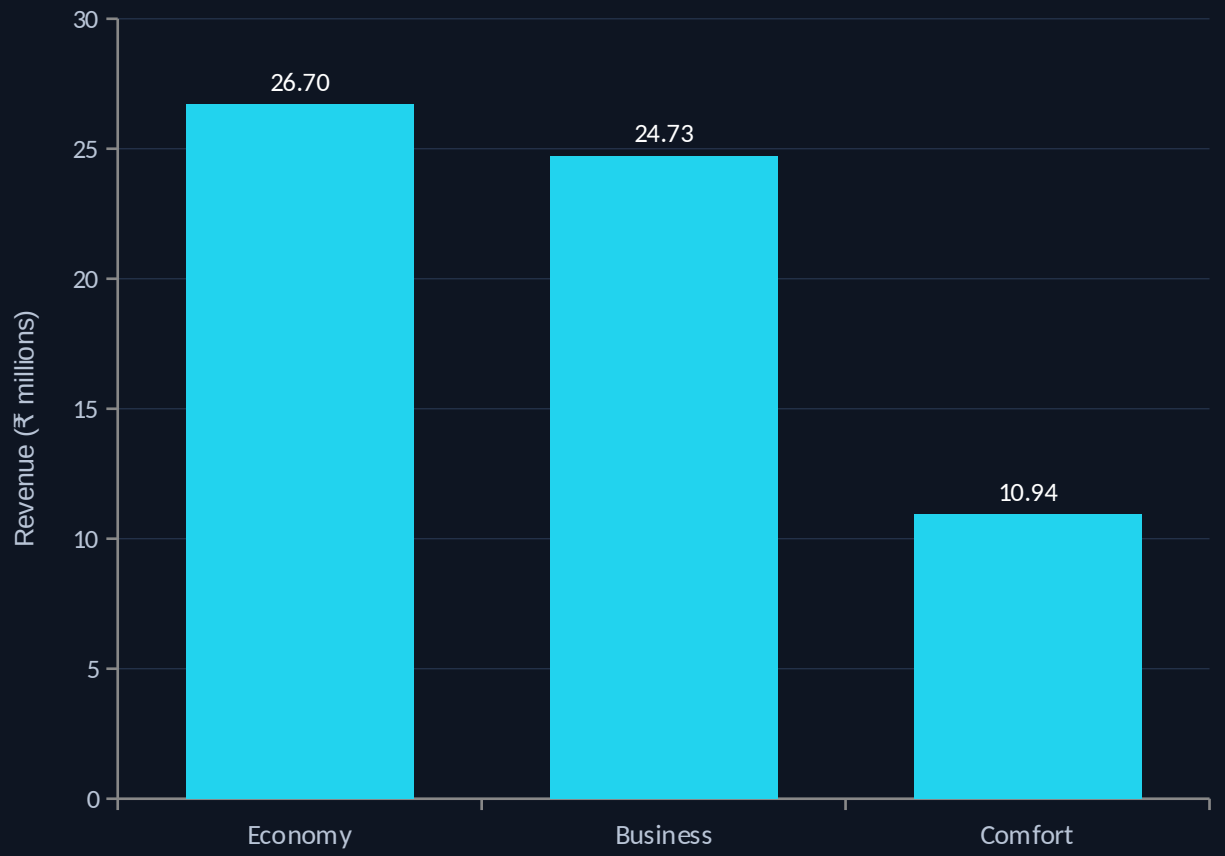
WHERE passenger_name LIKE 'A%'
  AND outbound=1;
```

Top 10 highest-priced segments

#	Ticket no	Flight ID	Fare class	Price (₹)
1	1000000000890	451	Business	59,969.57
2	1000000003532	165	Business	59,935.38
3	1000000004743	111	Business	59,918.54
4	1000000003748	24	Business	59,857.23
5	1000000003288	229	Business	59,799.31
6	1000000000949	140	Business	59,600.61
7	1000000004662	683	Business	59,512.14
8	1000000003485	793	Business	59,463.11
9	1000000000031	374	Business	59,432.50
10	1000000001412	159	Business	59,357.85

All ten most expensive segments are Business class, priced between ₹59,357 and ₹59,970.

Revenue by Fare Condition



Economy

4,653 segments (74.4%)

42.8% of revenue · avg ₹5,739 per segment

Business

662 segments (10.6%)

39.7% of revenue · avg ₹37,357 per segment

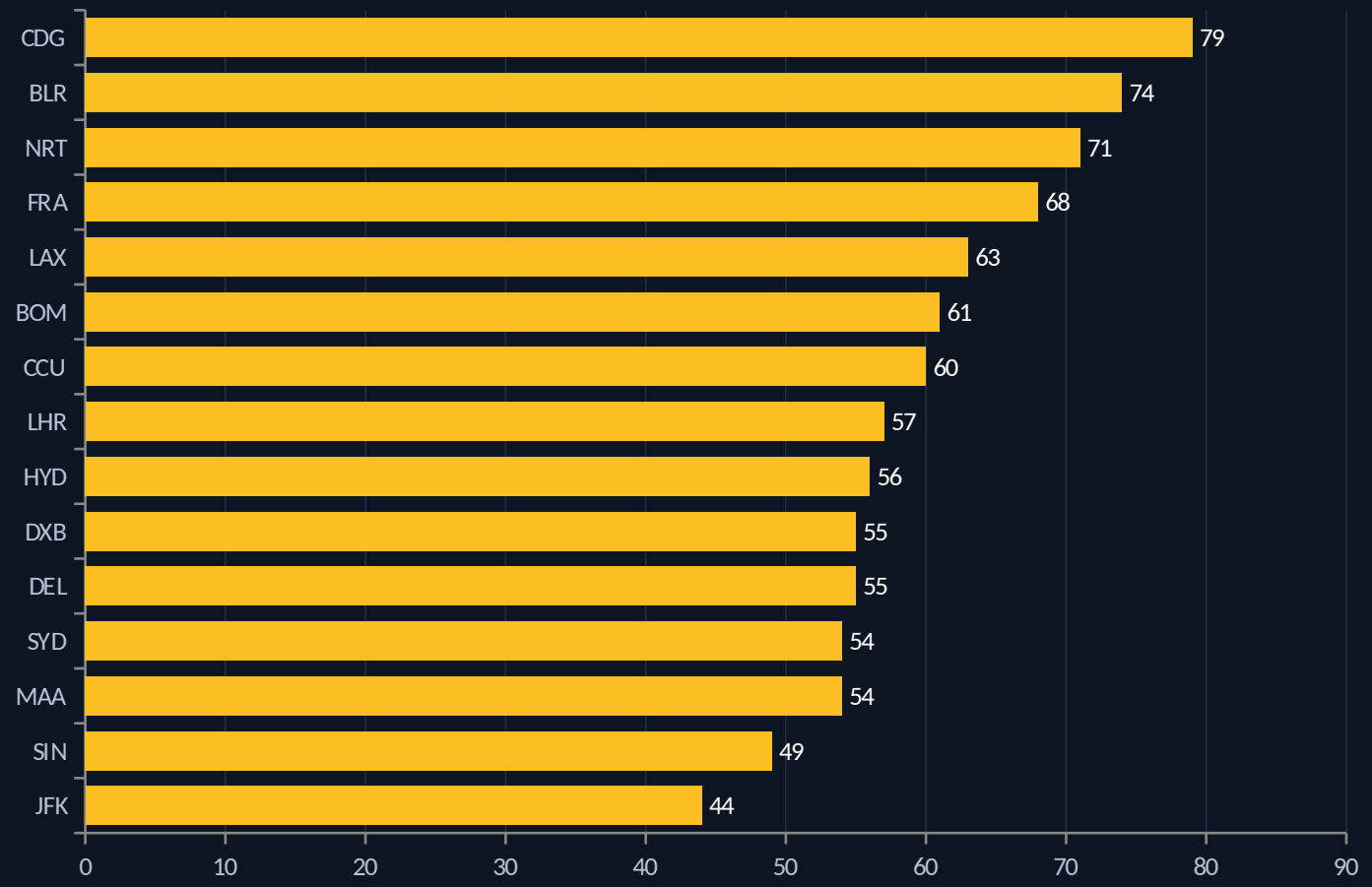
Comfort

943 segments (15.1%)

17.5% of revenue · avg ₹11,597 per segment

Business class is 10.6% of segments but 39.7% of revenue, averaging about 6.5x the Economy fare.

Departing Flights per Airport



SQL Logic Flow

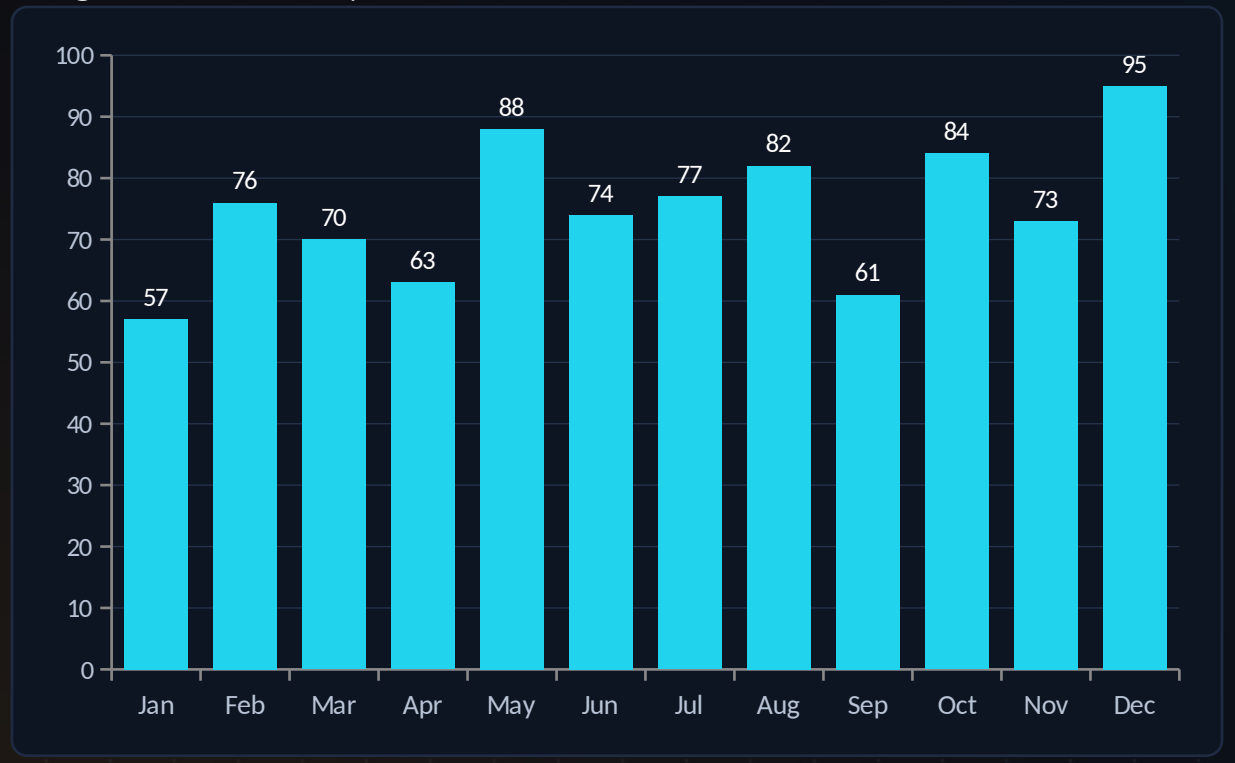
```
SELECT departure_airport,
  COUNT(*) AS flights
FROM flights
GROUP BY departure_airport
HAVING COUNT(*) > 40;
```

Paris CDG leads with 79 departures, followed by Bengaluru (74) and Tokyo Narita (71). All 15 airports clear the 40-flight threshold; the lowest is New York JFK with 44.

The HAVING > 40 filter keeps every airport, so a higher cutoff (such as 60) would separate the busiest hubs.

Flight Scheduling & Duration

Flights scheduled per month



Average scheduled duration by status (hours)



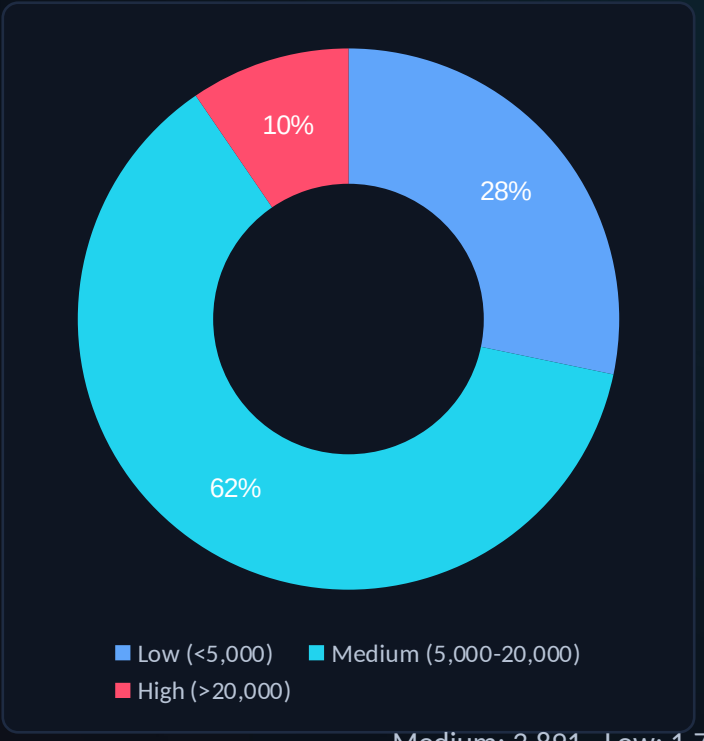
Passenger names average 13.3 characters (range 6 to 24), from UPPER() and LENGTH().

December is the busiest month (95 flights) and January the quietest (57). Durations cluster between 6.8 and 7.7 hours; "Departed" (n=13) is a small sample.

Transforming Data with CASE WHEN

```
SELECT ticket_no, flight_id, price,
 CASE
  WHEN price < 5000 THEN 'Low'
  WHEN price BETWEEN 5000 AND 20000
       THEN 'Medium'
  ELSE 'High'
 END AS price_category
FROM segments;
```

```
CASE WHEN seat_no IS NOT NULL
 THEN 'Checked_In'
 ELSE 'Not Checked In'
END AS check_in_status
```



Medium: 3,891 · Low: 1,770 · High: 597

3,083

Checked in (49.3%)

3,175

Not checked in (50.7%)

62% of segments are Medium priced. Seats are assigned only on Arrived and Departed flights, so check-in reflects flight progress, not passenger behavior.

Connecting the Database Through JOINs

JOINs combine related tables into richer analytical datasets.

FLIGHTS

flight_id
airplane_code
departure_airport

AIRPLANES

airplane_code
model
seat_capacity

```
flights.airplane_code = airplanes.airplane_code
```

SEGMENTS

ticket_no
flight_id
price

FLIGHTS ⋈ AIRPORTS

departure_airport
= airport_code
airport_name

```
segments.flight_id = flights.flight_id
```

Segment revenue by departure airport (₹M, top 10)



Bengaluru (BLR) generates the most segment revenue at ₹5.60M, ahead of Paris CDG (₹5.29M) despite CDG selling more segments (560 vs 524).

Top 5 Routes by Revenue



Rank	Route	Revenue (₹)	Gap vs #1
1	FRA → SIN	778,367.30	0
2	SIN → CCU	722,141.79	-56,226
3	DEL → BOM	708,206.58	-70,161
4	CCU → LAX	705,259.87	-73,107
5	CDG → FRA	700,885.02	-77,482

```
SELECT f.departure_airport, f.arrival_airport,
  ROUND(SUM(s.price),2) AS total_revenue
FROM flights f JOIN segments s USING (flight_id)
GROUP BY 1, 2 ORDER BY total_revenue DESC
LIMIT 5;
```

Frankfurt → Singapore is the top route at ₹778K; the top five routes sit within ₹78K of each other.

Total Booking Amount by Month



₹147.91M

Total booking amount, 2025

Mar · ₹13.92M

Peak month

Feb · ₹10.66M

Lowest month

Monthly booking value stays in a ₹10.7M to ₹13.9M band, with peaks in March, July and December.

Benchmarking with Subqueries

Bookings

Above-average total amount

Benchmark: ₹49,302.45

1,525

of 3,000 bookings (50.8%)

```
WHERE total_amount >
 (SELECT AVG(total_amount)
  FROM bookings)
```

Airplanes

Above-average seat capacity

Benchmark: 213.90 seats

5

of 10 aircraft: 777-300ER, A350-1000, A330-900neo, 787-9, A321neo

```
WHERE seat_capacity >
 (SELECT AVG(seat_capacity)
  FROM airplanes)
```

Segments

Above Business-class average

Benchmark: ₹37,356.86

311

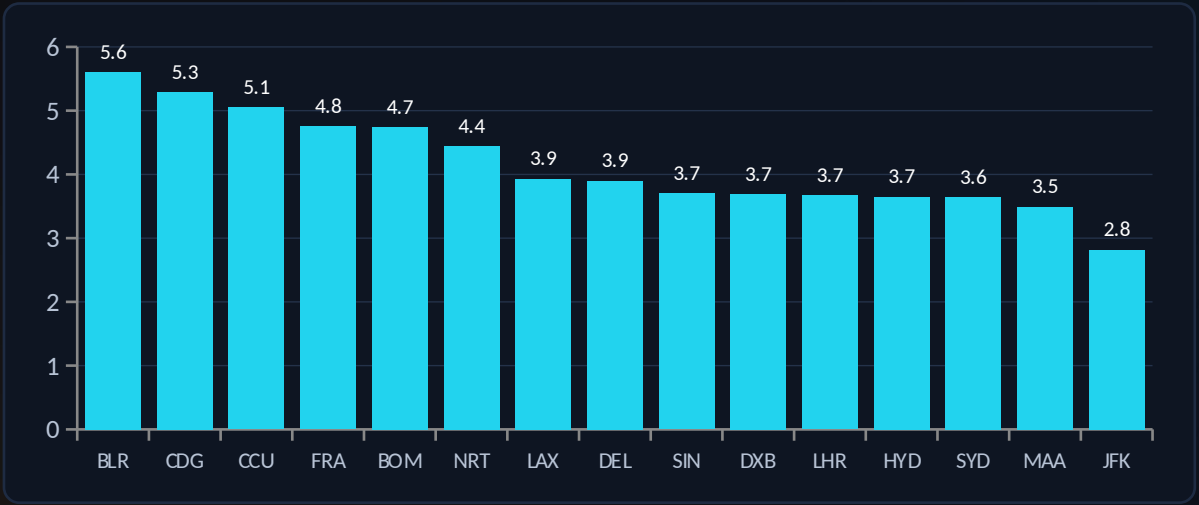
of 6,258 segments (5.0%)

```
WHERE price >
 (SELECT AVG(price)
  FROM segments
  WHERE fare_conditions='Business')
```

Subqueries compute the benchmark once and compare every row against it, with no hard-coded thresholds.

CTEs and Views

Revenue by departure airport (₹M): all 15 clear the ₹1M threshold



1 JOIN
segments ⋈ flights
fare linked to departure airport

2 CTE
airport_stats
COUNT and SUM per airport

3 JOIN
stats ⋈ airports
attach the airport name

4 FILTER
revenue > ₹1,000,000
sorted high to low

Common Table Expression: airport revenue > ₹1,000,000

Airport	Segments	Revenue (₹)
Kempegowda International	524	5,604,260
Charles de Gaulle	560	5,293,141
Netaji Subhas Chandra Bose Intl	463	5,051,931
Frankfurt Airport	496	4,750,889
Chhatrapati Shivaji Maharaj Intl	439	4,737,511

All 15 airports qualify; top 5 shown.

View: high_value_bookings

```
CREATE VIEW ... WHERE total_amount >= 50000
```

Returns 1,509 bookings as a reusable virtual table.

The CTE splits aggregation from the airport join, making the logic readable and reusable.

Window Functions

RANK()

Rank segments within each flight by price

```
RANK() OVER (PARTITION BY flight_id
  ORDER BY price DESC)
```

Sample: Flight 121

Ticket (last 4)	Price	Rank
1290	47,932.61	1
4180	17,351.56	2
0447	10,694.90	3
3706	9,304.49	4

LAG()

Previous segment price on the same ticket

```
LAG(price) OVER (PARTITION BY ticket_no
  ORDER BY scheduled_departure)
```

Sample: 1,453 round-trip tickets

Ticket (last 4)	Price	Previous
0002 (leg 1)	3,414.06	NULL
0002 (leg 2)	5,145.47	3,414.06
0004 (leg 2)	13,480.37	14,569.10
0016 (leg 2)	6,251.00	8,205.54

ROW_NUMBER()

Sequence flights per airplane by departure

```
ROW_NUMBER() OVER (PARTITION BY
airplane_code
  ORDER BY scheduled_departure)
```

Sample: Airplane 32N

Flight	Departure	Seq
AI145	07 Jan 07:00	1
AI945	15 Jan 02:00	2
AI255	26 Jan 17:00	3
AI732	05 Feb 07:30	4



Window functions keep every row while adding rank, previous-value and sequence columns, which GROUP BY cannot do.

KEY INSIGHTS

01

Revenue Performance

₹62.37M

Segment revenue across 6,258 legs, averaging ₹9,966.61 per segment.

02

Fare Class Value

39.7%

of revenue comes from Business class, which is only 10.6% of segments.

03

Airport & Route Leaders

BLR · ₹5.60M

Bengaluru tops airport revenue; Frankfurt → Singapore tops routes at ₹778K.

Overall: SQL turns raw flight, ticket and fare records into measurable indicators of revenue, pricing and network performance.

Comparative Analysis

Rank	Airport	City	Segments	Revenue (₹)	Gap vs #1 (₹)
1	BLR	Bengaluru	524	5,604,260	0
2	CDG	Paris	560	5,293,141	-311,119
3	CCU	Kolkata	463	5,051,931	-552,329
4	FRA	Frankfurt	496	4,750,889	-853,371
5	BOM	Mumbai	439	4,737,511	-866,749

BLR · ₹5,604,260

Highest: Bengaluru

JFK · ₹2,821,742

Lowest: New York JFK (289 segments)

Bengaluru earns nearly twice the revenue of New York JFK; the #5 airport (Mumbai) trails the leader by ₹866,749.

CONCLUSION

Conclusion & Future Scope

Project Conclusion

Cleaned 7 tables in Python (Jupyter), then analyzed them in MySQL.

Identified patterns in fares, revenue, airports, routes and scheduling.

Applied joins, subqueries, CTEs, views and window functions.

Note: booking totals (₹147.9M) and segment fares (₹62.4M) are recorded independently and do not reconcile; segment price is used for revenue.

Future Scope

Build real-time performance dashboards.

Add delay and cancellation analysis (3.9% delayed, 2.7% cancelled).

Forecast demand by route and month.

Segment passengers and optimize fare-class pricing.

Automate reporting and alerts.

THANK YOU

Ananya Gadodia
Data Analytics | MySQL · SQL Project

